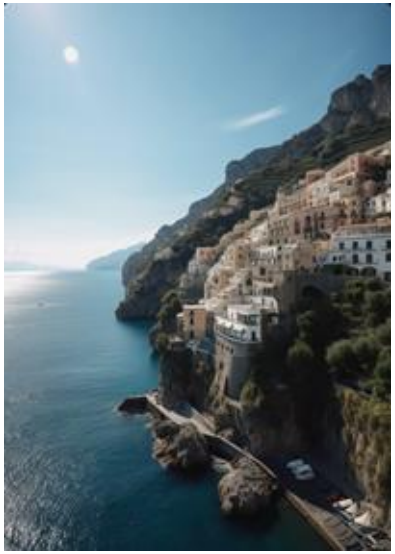
Generative Models

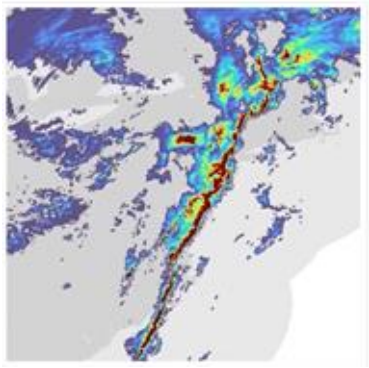
1. Introduction to Generative Models
2. GANs
3. VAE
4. Physical intuition
5. Diffusion Models
6. Score-based Generative Models
7. Conditional Flow Matching
8. Controllable Generation - CFG

Recent progress of generative models

Black and White Portrait of a young blond woman with a shirt and a tatoo on the neck looking at the camera.

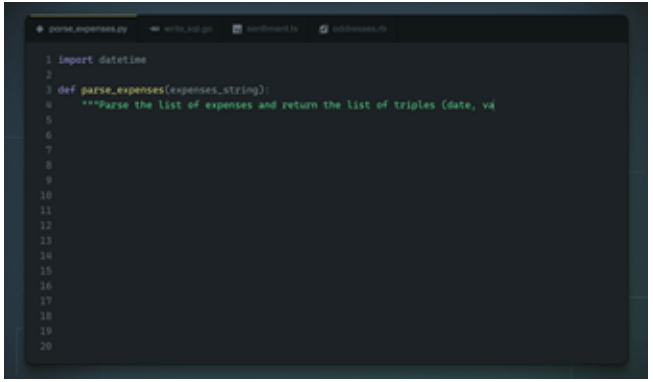


Recent progress of generative models



Weather Nowcasting

Text, Code Generation



Landscape of Deep Generative Learning

What is Generative Modelling ?

The goal are twofold:

What is Generative Modelling ?

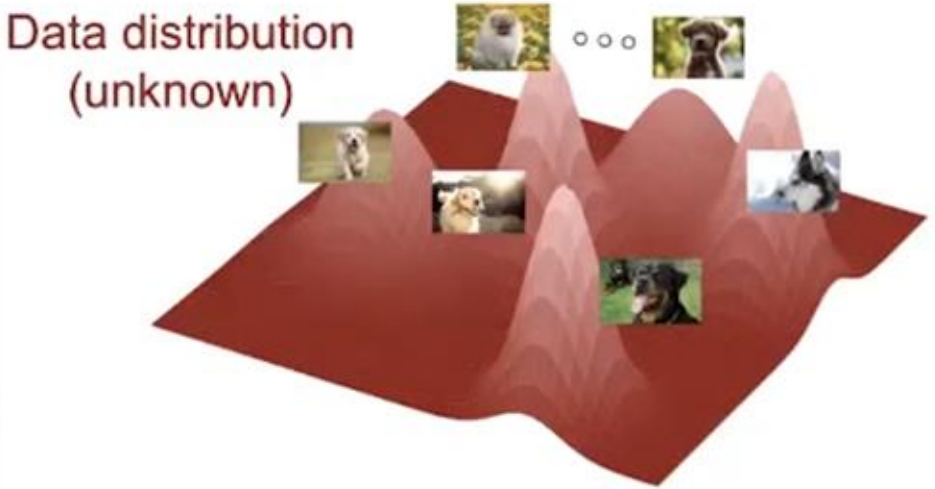



○ ○ ○

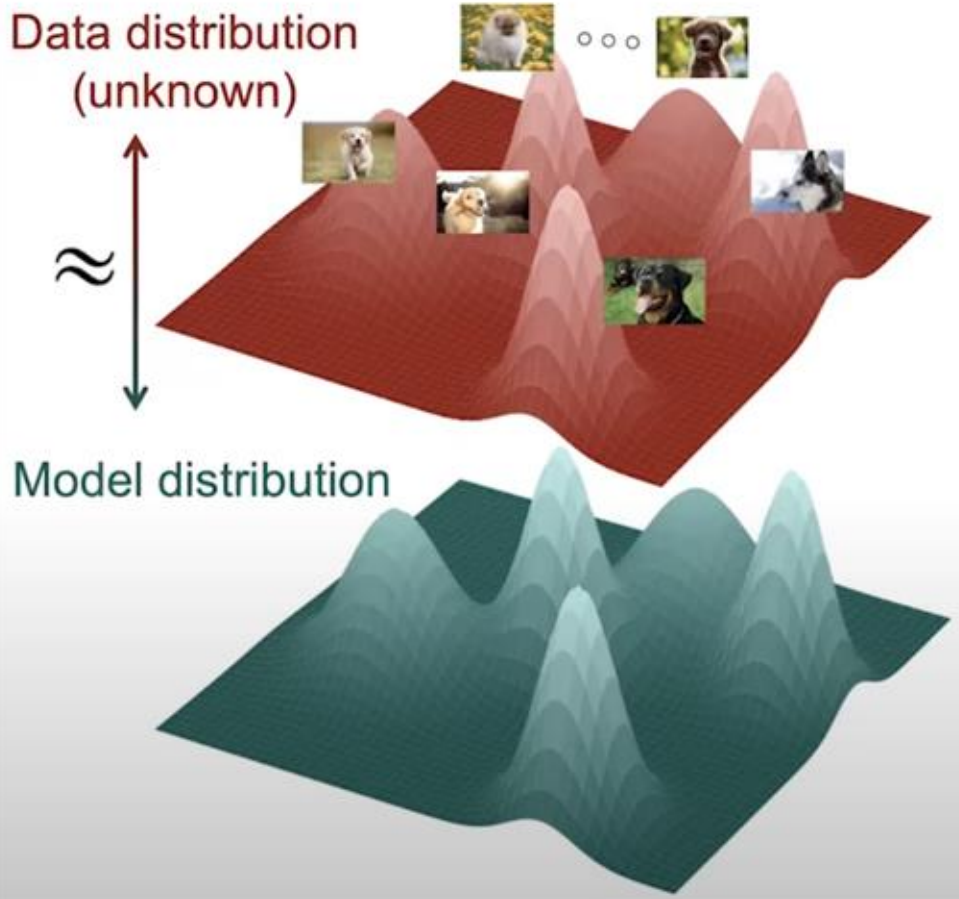



Large collection of data samples

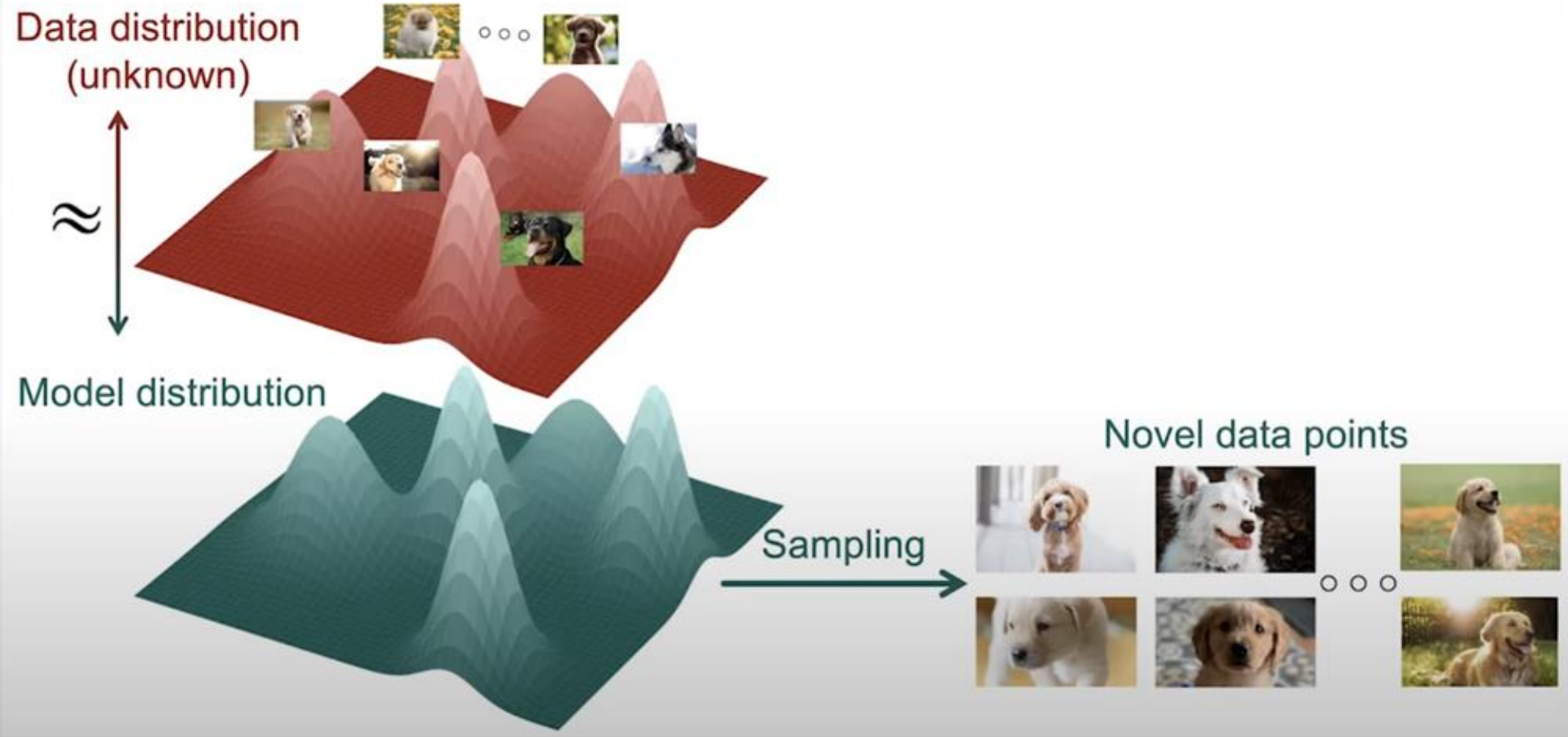
Estimating the probability distribution of data



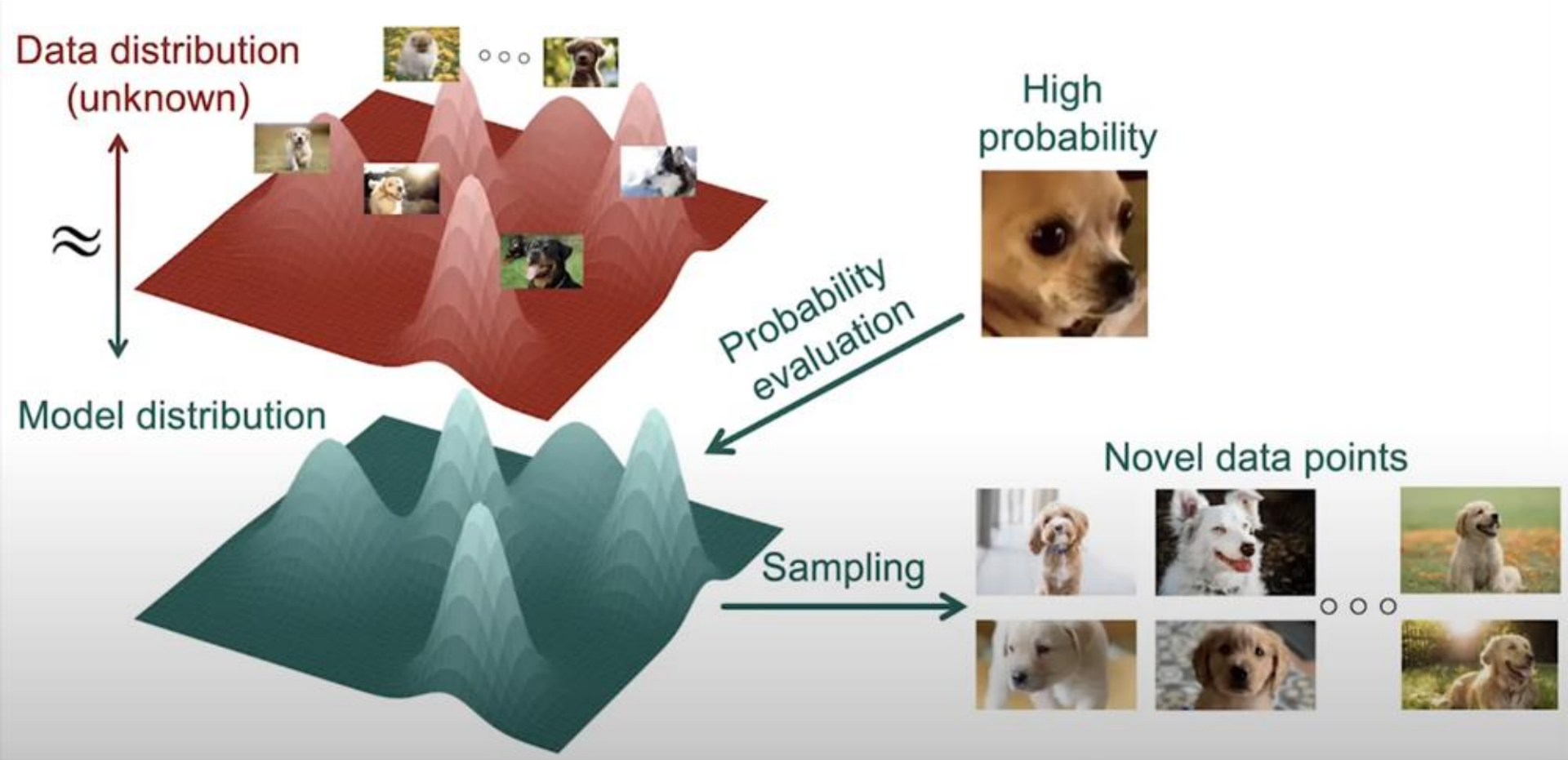
Estimating the probability distribution of data



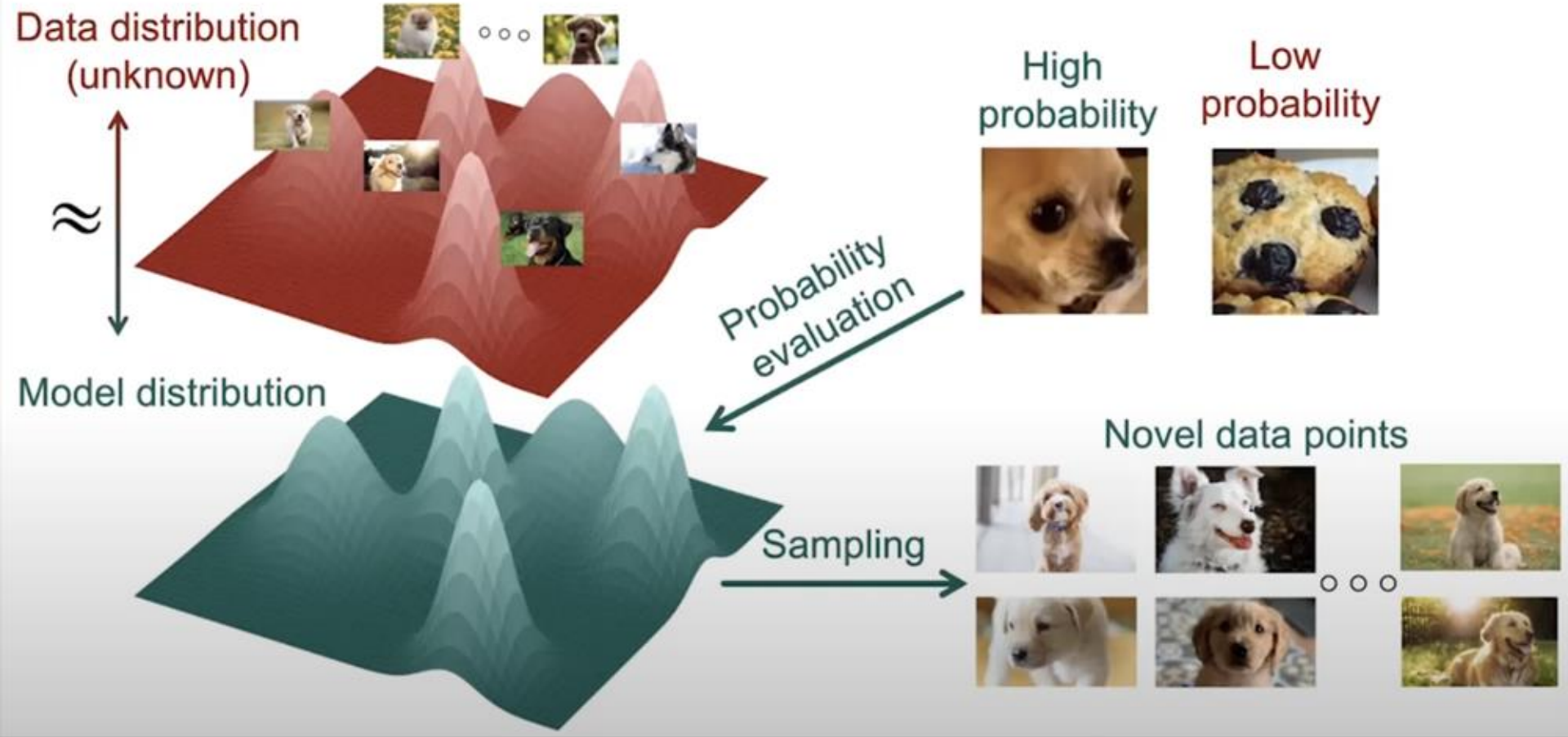
Estimating the probability distribution of data



Estimating the probability distribution of data



Estimating the probability distribution of data

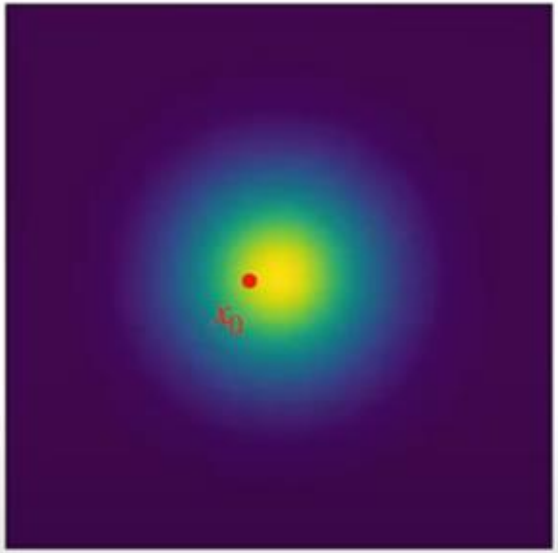


Generative models - Sampling

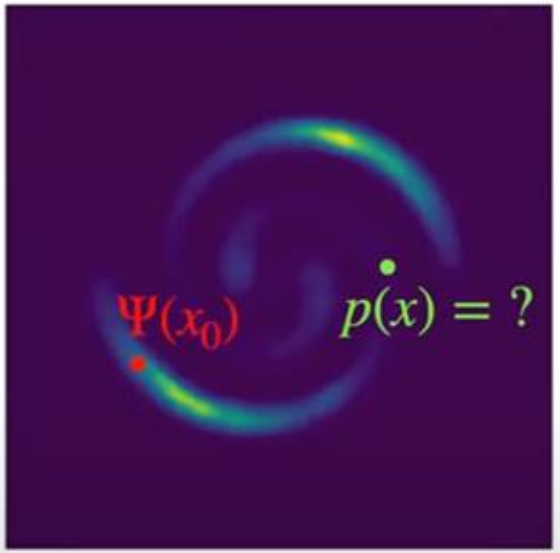
$p(x)$

$q(x)$

$\mathbb{R}^d$



Ψ



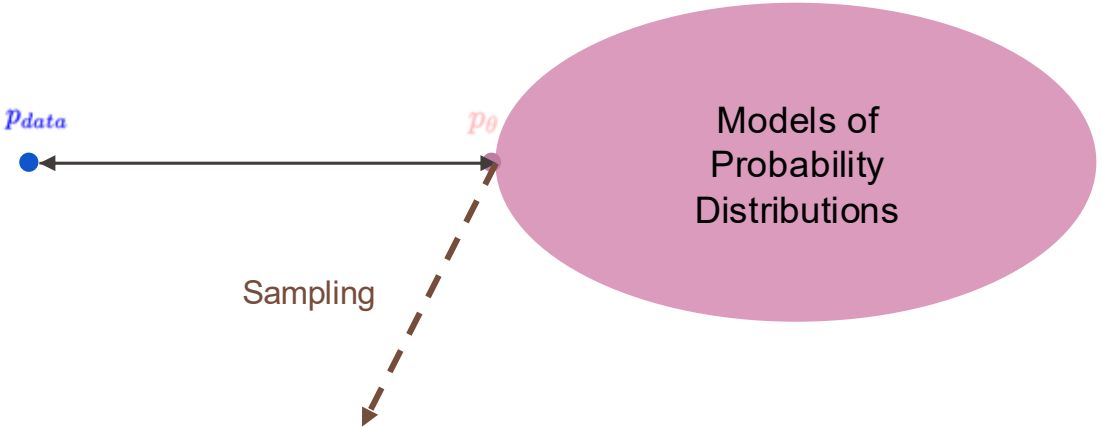
$x_0 \sim p$

$\Psi(x_0) \sim q$

Training generative models

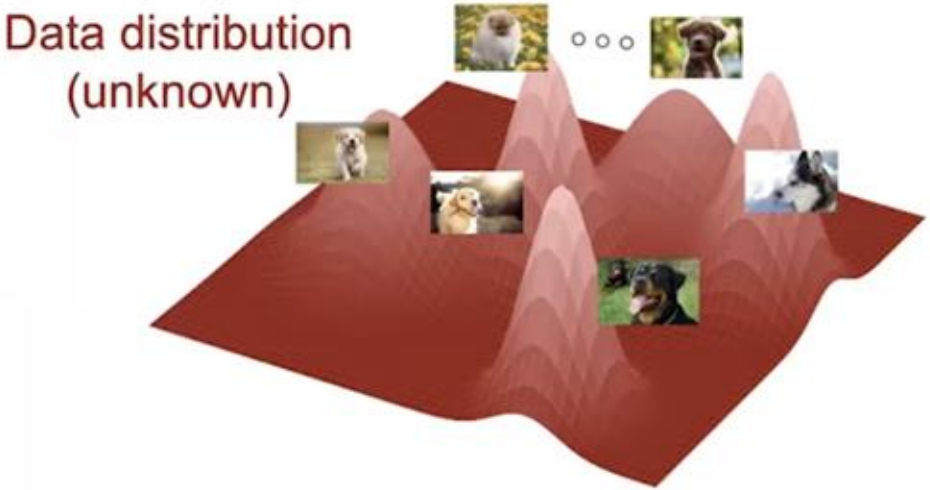



$$\boldsymbol{X}_i \overset{i.i.d.}{\sim} p_{data}$$
$$i = 1, 2, ..., N$$



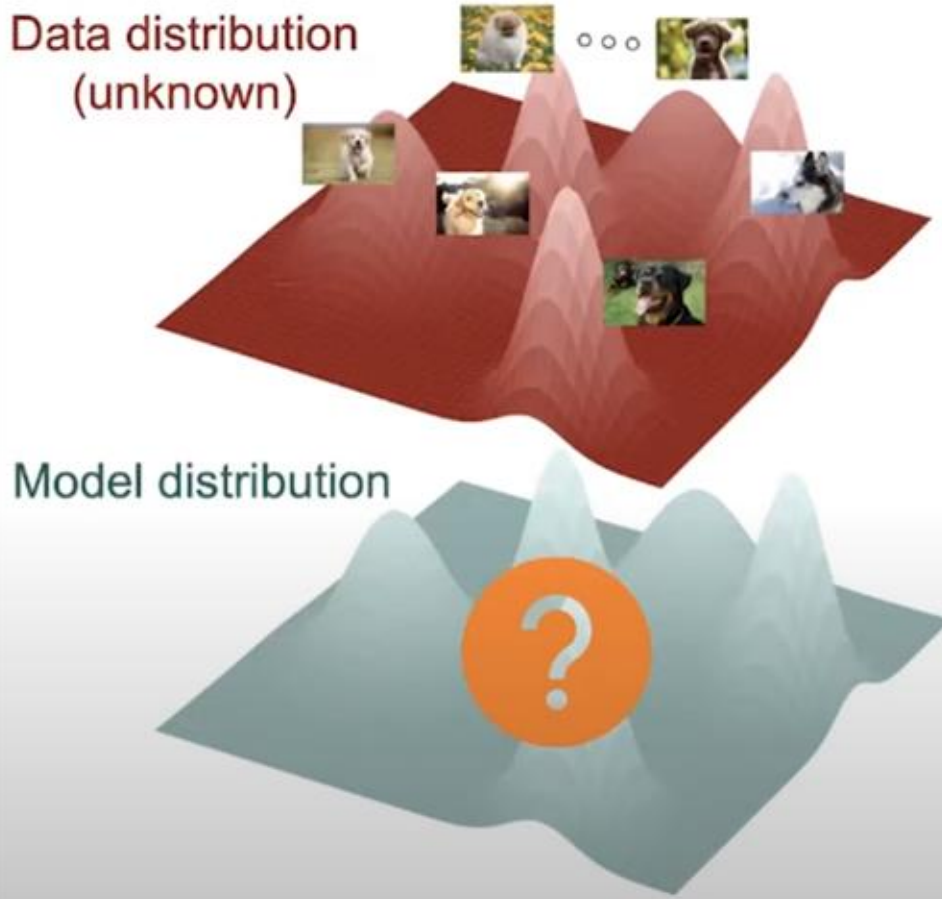


Key challenge



Data distribution is extremely complex for high dimensional data.

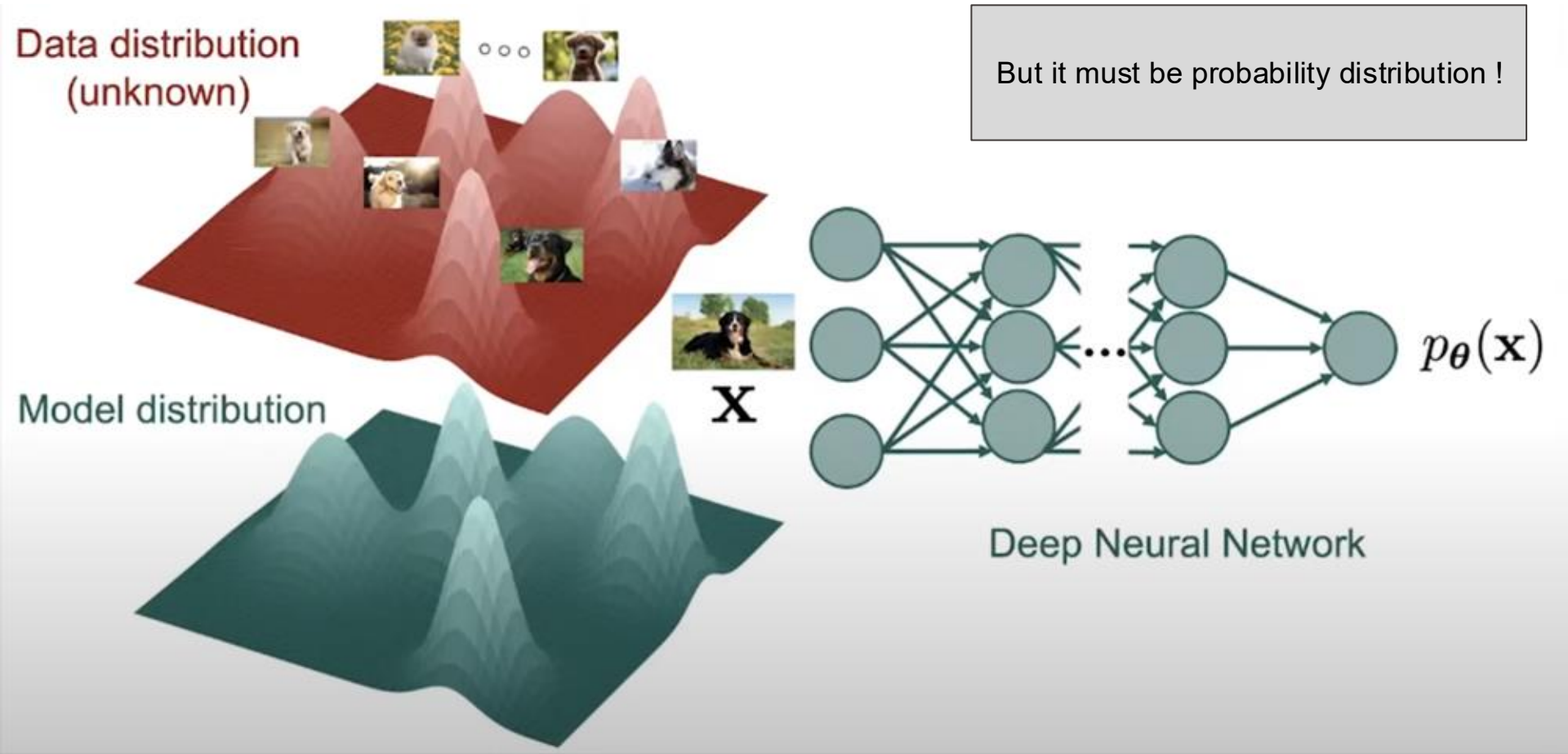
Key challenge



Data distribution is extremely complex for high dimensional data.

How to build a complex model to fit the data distribution?

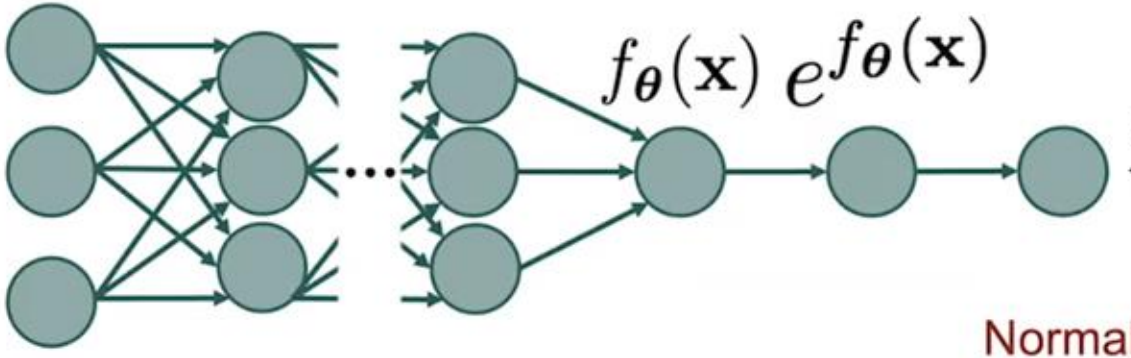
Key challenge



But it must be probability distribution !

Key challenge

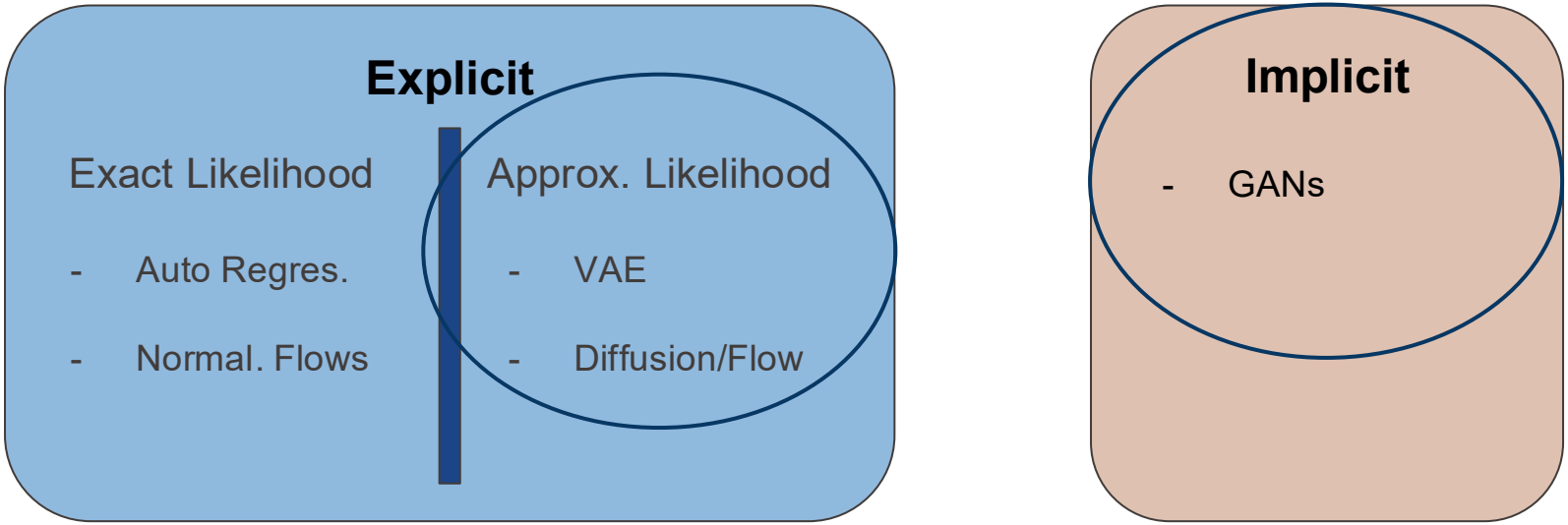
$\mathbf{x}$



$Z_{\theta} =$?

Taxonomy of Modelling

A fundamental distinction lies in how the models parametrize the underlying data distribution, whether $p_\theta(x)$ is specified explicitly or implicitly.



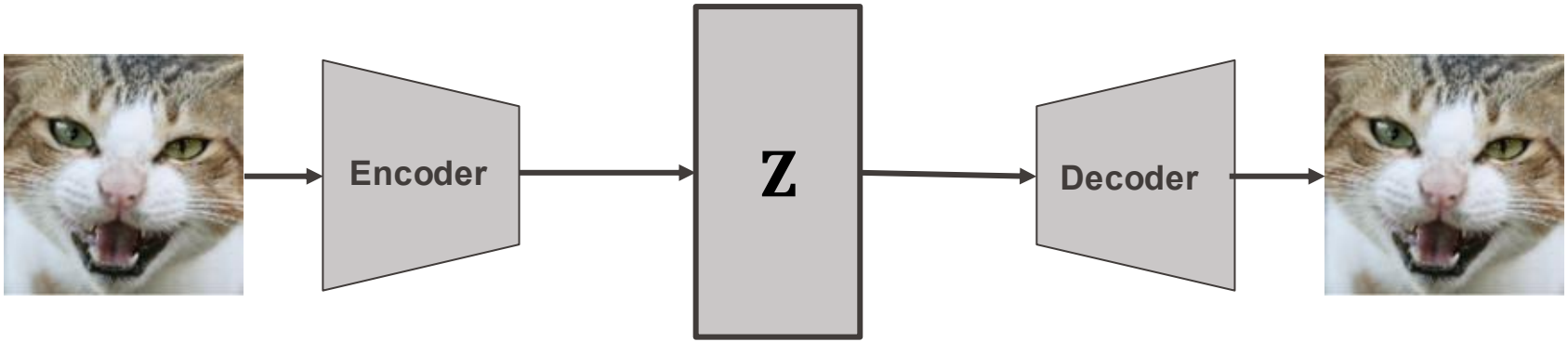
GANs

Generative Adversarial Networks

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))].$$

Variational AutoEncoder (VAE)

AutoEncoder



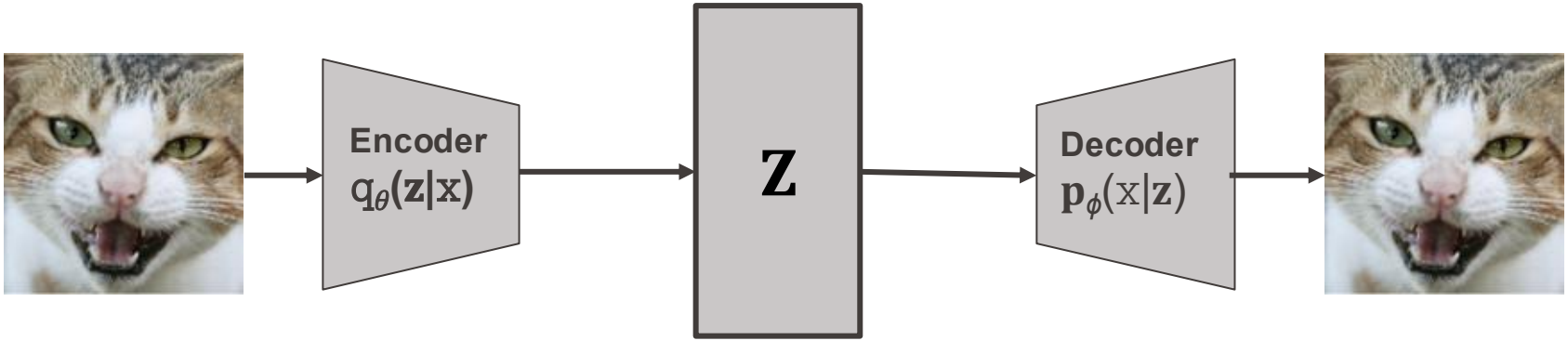
- We add constraint on **Z** (Dimensionality reduction, Structural constraint, …)
- We train using a reconstruction loss:

$$Loss = ||Decoder(Encoder(x)) - x||^2$$

- For Denoising AutoEncoder, we train it to be robust to small perturbation:

$$Loss = ||Decoder(Encoder(x + \epsilon)) - x||^2, \epsilon \sim \mathcal{N}(0, \sigma)$$

Variational AutoEncoder - p(z) ~ N(0,Id)



For any data point $\mathbf{x}$, the log-likelihood satisfies:

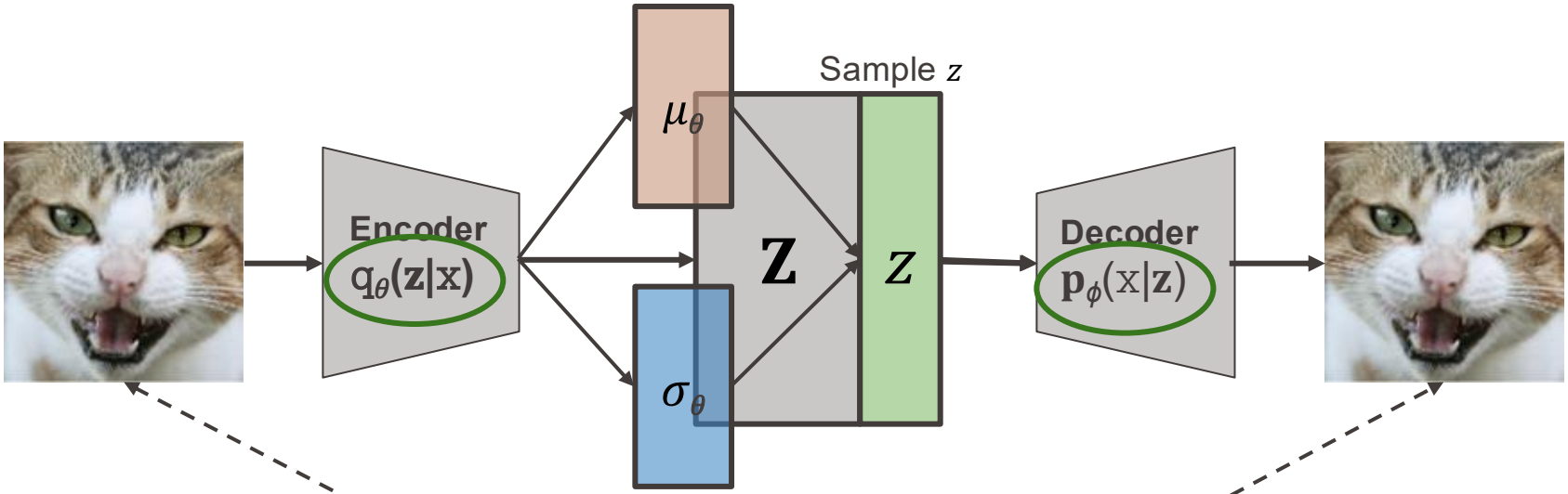
$$\log p_{\boldsymbol{\phi}}(\mathbf{x}) \geq \mathcal{L}_{\mathrm{ELBO}}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{x}),$$

where the ELBO is given by:

$$\mathcal{L}_{\mathrm{ELBO}} = \underbrace{\mathbb{E}_{\mathbf{z}\sim q_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x})}\left[\log p_{\boldsymbol{\phi}}(\mathbf{x}|\mathbf{z})\right]}_{\text{Reconstruction Term}} - \underbrace{\mathcal{D}_{\mathrm{KL}}\left(q_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z})\right)}_{\text{Latent Regularization}}.$$

The Kullback-Leibler divergence, $\mathcal{D}_{KL}$ is a common statistical distance between probability distributions.

Variational AutoEncoder



1. Reconstruction Loss

Minimize the Distance

$$\mathbf{z} \sim p_{\text{prior}} := \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

$$q_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x}) := \mathcal{N}\left(\mathbf{z}; \boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}^2_{\boldsymbol{\theta}}(\mathbf{x}))\right),$$

$$p_{\boldsymbol{\phi}}(\mathbf{x}|\mathbf{z}) := \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_{\boldsymbol{\phi}}(\mathbf{z}), \sigma^2\mathbf{I}),$$

Variational AutoEncoder - Pseudo Code


```
# 1. ENCODE: Get the mean and log-variance
mu, logvar = model.encode(x)
```

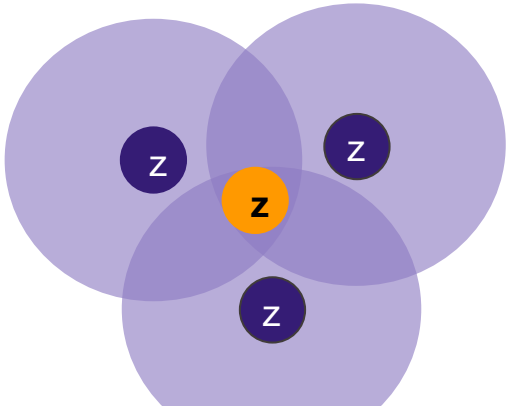
```
# 2. SAMPLE (Reparameterize): Get z
std = torch.exp(0.5 * logvar)
eps = torch.randn_like(std)
z = mu + eps * std
```

Reparametrization Trick

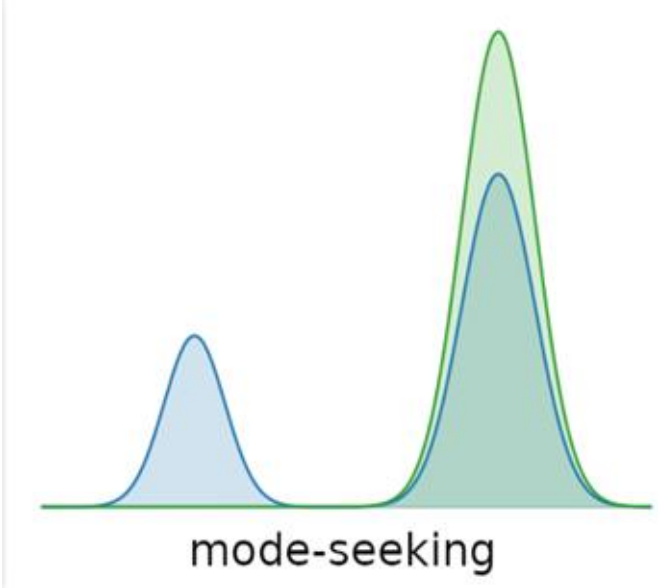
```
# 3. DECODE: Reconstruct the image
x_pred = model.decode(z)
```

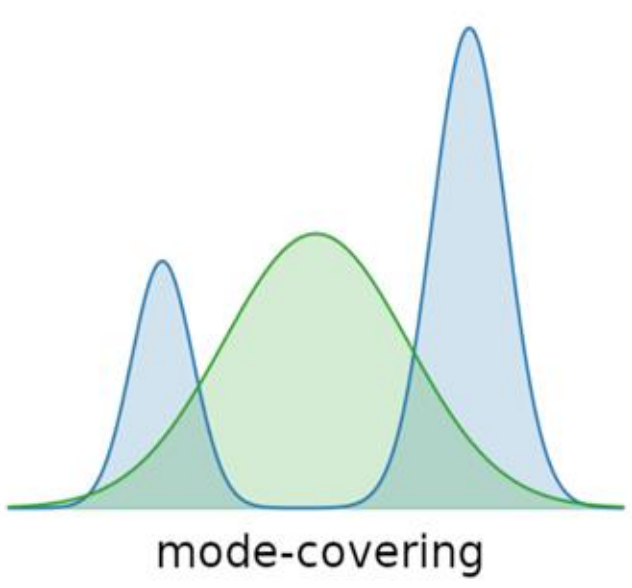
```
# --- THE LOSS ---
recon_loss = F.mse_loss(x_pred, x, reduction='sum')
kl_loss = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
total_loss = recon_loss + kl_loss
```

Variational AutoEncoder - Drawback



Perceptual vs Likelihood



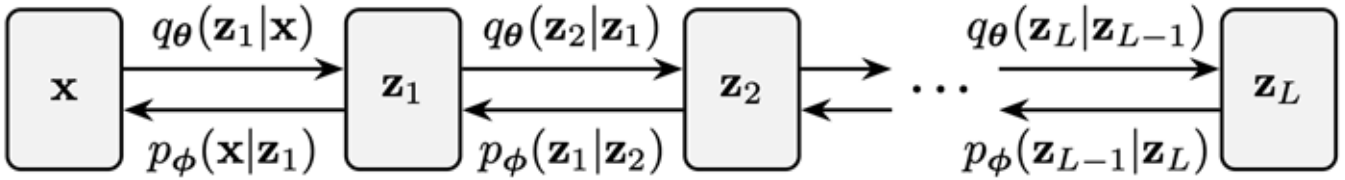


GANs

Likelihood-based (VAE, …)

Variational AutoEncoder - Limits

Hierarchical VAE



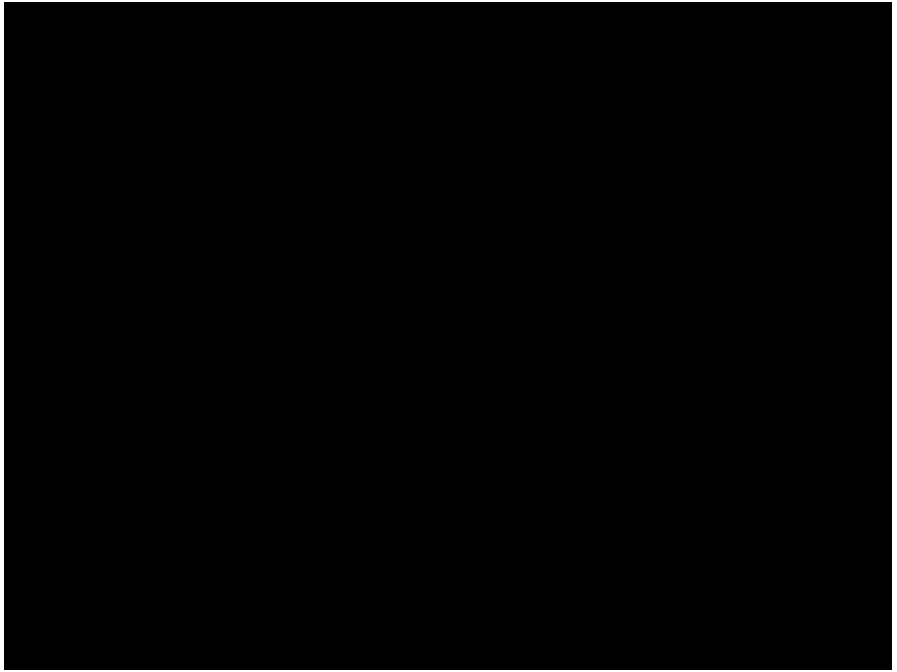
- **Problem**: Their training poses a unique challenge, because encoder and decoder are optimized jointly, learning becomes unstable. Gradient information is often weak in deeper layers making it difficult for them to contribute meaningfully.

Diffusion Models

Physical Intuition

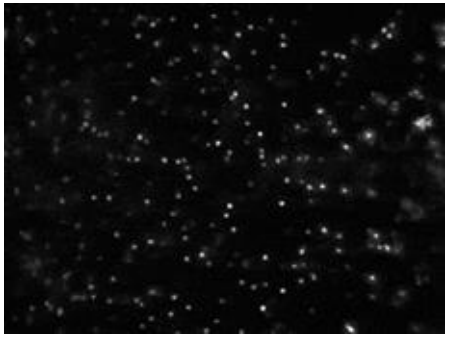
- Destroy Structure in Data
- Carefully characterize the destruction
- Learn how to reverse time

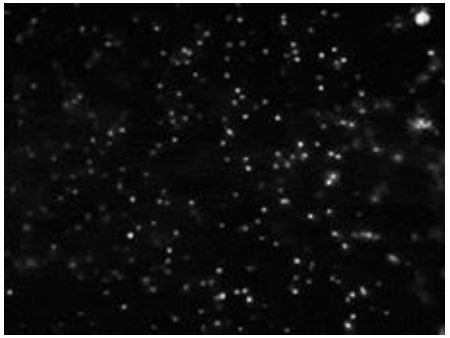
Observation 1 : Diffusion destroys the structure in the data



Core Idea : Recover Structure by Reversing Time

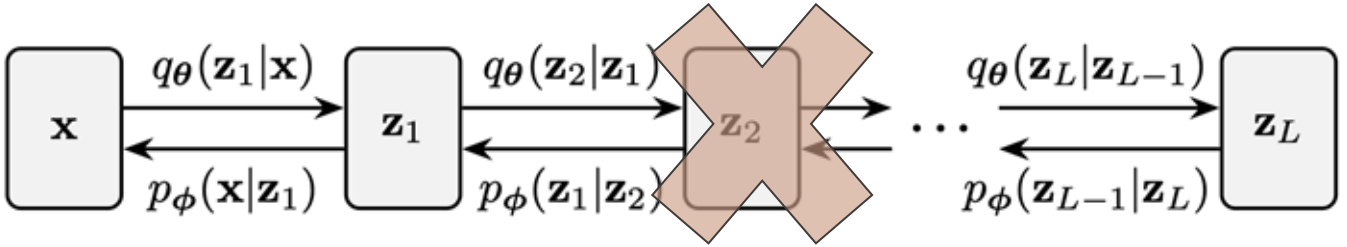
Observation 2 : Microscopic diffusion is time reversible



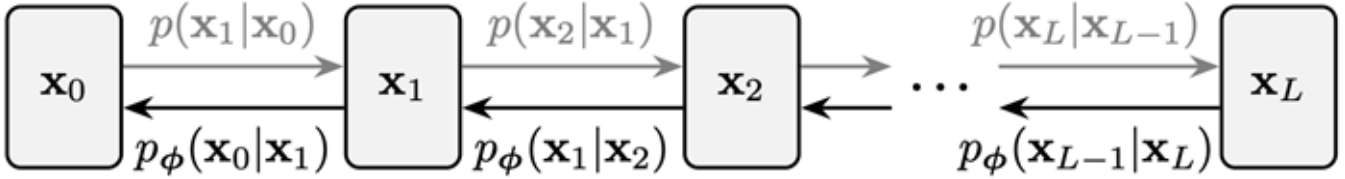


Overview : Diffusion Probabilistic Models

Hierarchical VAE → Diffusion Models



Fixed Encoder



Denoising Diffusion Probabilistic Models - Definition

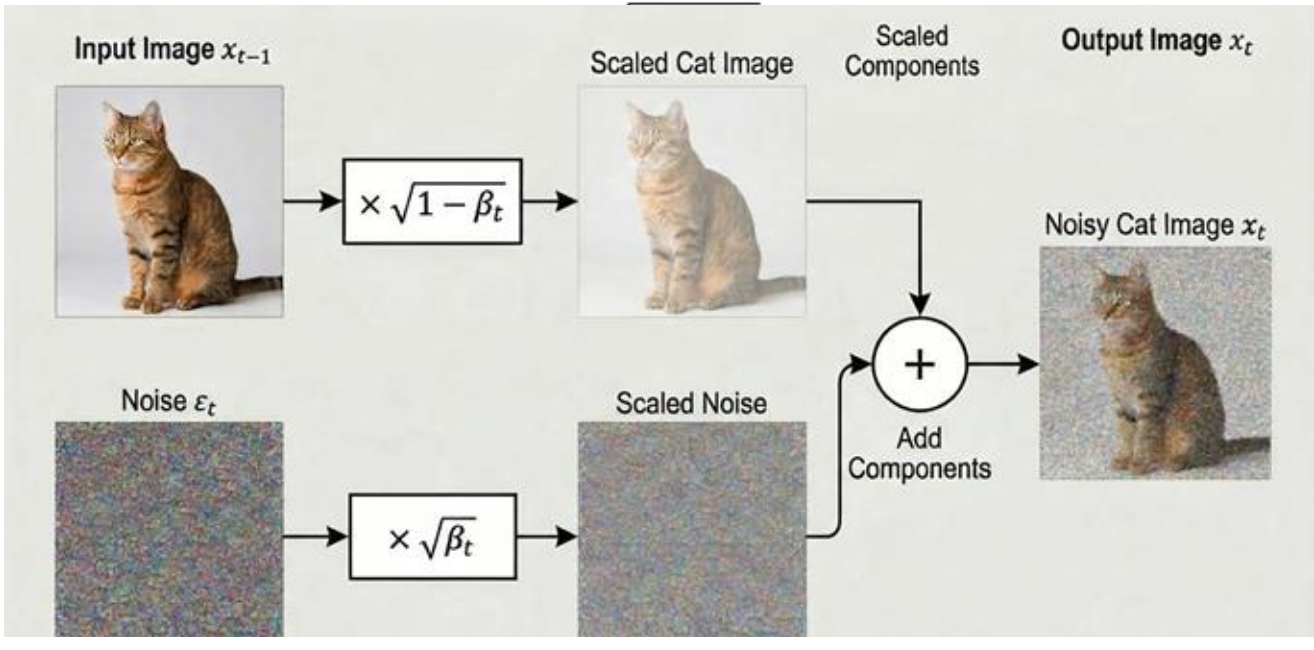
[Johnatan Ho et al., NeurIPS 2020][Jascha Sohl-Dickstein et al., ICLR 2015]

Data Distribution

Forward Diffusion

Noise Distribution

$q(\mathbf{x}_0)$ → $q(\mathbf{x}_T) \simeq \mathcal{N}(\mathbf{x}_T; 0, \mathbf{I})$



Denoising Diffusion Probabilistic Models

[Johnatan Ho et al., NeurIPS 2020]

Data Distribution

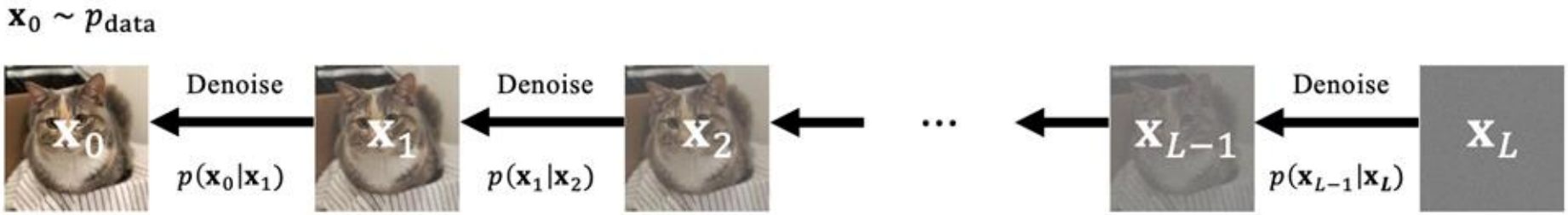
Reverse Diffusion

Noise Distribution

$$p(\mathbf{x}_0) \simeq q(\mathbf{x}_0) \qquad p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$$

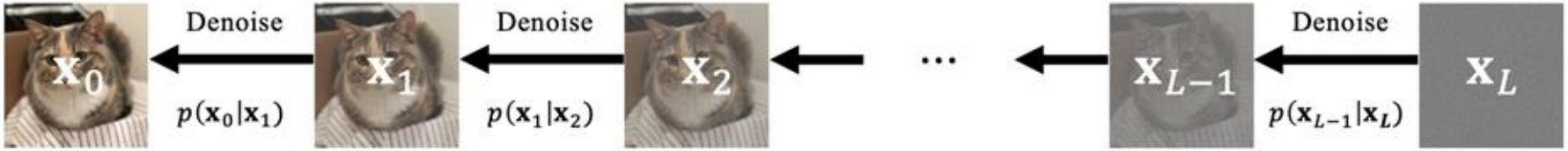
$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) := \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t))$$

Learned drift and covariance functions



Denoising Diffusion Probabilistic Models - Training

$\mathbf{x}_0 \sim p_{\text{data}}$



Denoising Diffusion Probabilistic Models - Training

$$p(\mathbf{x}_{i-1}|\mathbf{x}_i) = p(\mathbf{x}_i|\mathbf{x}_{i-1}) \underbrace{\frac{p_{i-1}(\mathbf{x}_{i-1})}{p_i(\mathbf{x}_i)}}_{\text{intractable}}. \quad \Rightarrow \quad p(\mathbf{x}_{i-1}|\mathbf{x}_i, \mathbf{x}) = p(\mathbf{x}_i|\mathbf{x}_{i-1}) \frac{p(\mathbf{x}_{i-1}|\mathbf{x})}{p(\mathbf{x}_i|\mathbf{x})}.$$

The following equality holds:

$$\begin{aligned} &\mathbb{E}_{p_i(\mathbf{x}_i)} \left[\mathcal{D}_{\mathrm{KL}} \left(p(\mathbf{x}_{i-1}|\mathbf{x}_i) \| p_{\boldsymbol{\phi}}(\mathbf{x}_{i-1}|\mathbf{x}_i) \right) \right] \\ &= \mathbb{E}_{p_{\mathrm{data}}(\mathbf{x})} \mathbb{E}_{p(\mathbf{x}_i|\mathbf{x})} \left[\mathcal{D}_{\mathrm{KL}} \left(p(\mathbf{x}_{i-1}|\mathbf{x}_i, \mathbf{x}) \| p_{\boldsymbol{\phi}}(\mathbf{x}_{i-1}|\mathbf{x}_i) \right) \right] + C, \end{aligned}$$

$$p(\mathbf{x}_{i-1}|\mathbf{x}_i, \mathbf{x}) = \mathcal{N}\left(\mathbf{x}_{i-1}; \boldsymbol{\mu}\left(\mathbf{x}_i, \mathbf{x}, i\right), \sigma^2(i)\mathbf{I} \right),$$

Denoising Diffusion Probabilistic Models - Training

$$p(\mathbf{x}_{i-1}|\mathbf{x}_i, \mathbf{x}) = \mathcal{N}\left(\mathbf{x}_{i-1}; \boldsymbol{\mu}\left(\mathbf{x}_i, \mathbf{x}, i\right), \sigma^2(i)\mathbf{I}\right),$$

where

$$\boldsymbol{\mu}\left(\mathbf{x}_i, \mathbf{x}, i\right) := \frac{\bar{\alpha}_{i-1}\beta_i^2}{1-\bar{\alpha}_i^2}\mathbf{x} + \frac{(1-\bar{\alpha}_{i-1}^2)\alpha_i}{1-\bar{\alpha}_i^2}\mathbf{x}_i, \quad \sigma^2(i) := \frac{1-\bar{\alpha}_{i-1}^2}{1-\bar{\alpha}_i^2}\beta_i^2.$$

$$\sum_{i=1}^{L} \mathbb{E}_{p(\mathbf{x}_i|\mathbf{x}_0)}\left[\mathcal{D}_{\mathrm{KL}}\left(p(\mathbf{x}_{i-1}|\mathbf{x}_i, \mathbf{x}_0) \| p_{\boldsymbol{\phi}}(\mathbf{x}_{i-1}|\mathbf{x}_i)\right)\right].$$

$$\mathcal{L}_{\mathrm{diffusion}}(\mathbf{x}_0; \boldsymbol{\phi}) = \sum_{i=1}^{L} \frac{1}{2\sigma^2(i)} \|\boldsymbol{\mu}_{\boldsymbol{\phi}}(\mathbf{x}_i, i) - \boldsymbol{\mu}(\mathbf{x}_i, \mathbf{x}_0, i)\|_2^2 + C,$$

Denoising Diffusion Probabilistic Models - Training

$$\mathcal{L}_{\text{diffusion}}(\mathbf{x}_0; \boldsymbol{\phi}) = \sum_{i=1}^{L} \frac{1}{2\sigma^2(i)} \left\| \boldsymbol{\mu}_{\boldsymbol{\phi}}(\mathbf{x}_i, i) - \boldsymbol{\mu}(\mathbf{x}_i, \mathbf{x}_0, i) \right\|_2^2 + C,$$

Denoising Diffusion Probabilistic Models - Training

```
x_0 = data[idx]
```

```
# 1. Sample random timesteps t for each data point in the batch
t = torch.randint(0, T, (batch_size,))
# 2. Sample random pure Gaussian noise epsilon
noise = torch.randn_like(x_0)
```

```
# 3. Forward diffusion: Calculate x_t using the closed-form formula
# x_t = sqrt(alpha_bar_t) * x_0 + sqrt(1 - alpha_bar_t) * noise
sqrt_alpha_bar_t = sqrt_alphas_cumprod[t].unsqueeze(-1)
sqrt_one_minus_alpha_bar_t = sqrt_one_minus_alphas_cumprod[t].unsqueeze(-1)

x_t = sqrt_alpha_bar_t * x_0 + sqrt_one_minus_alpha_bar_t * noise
```

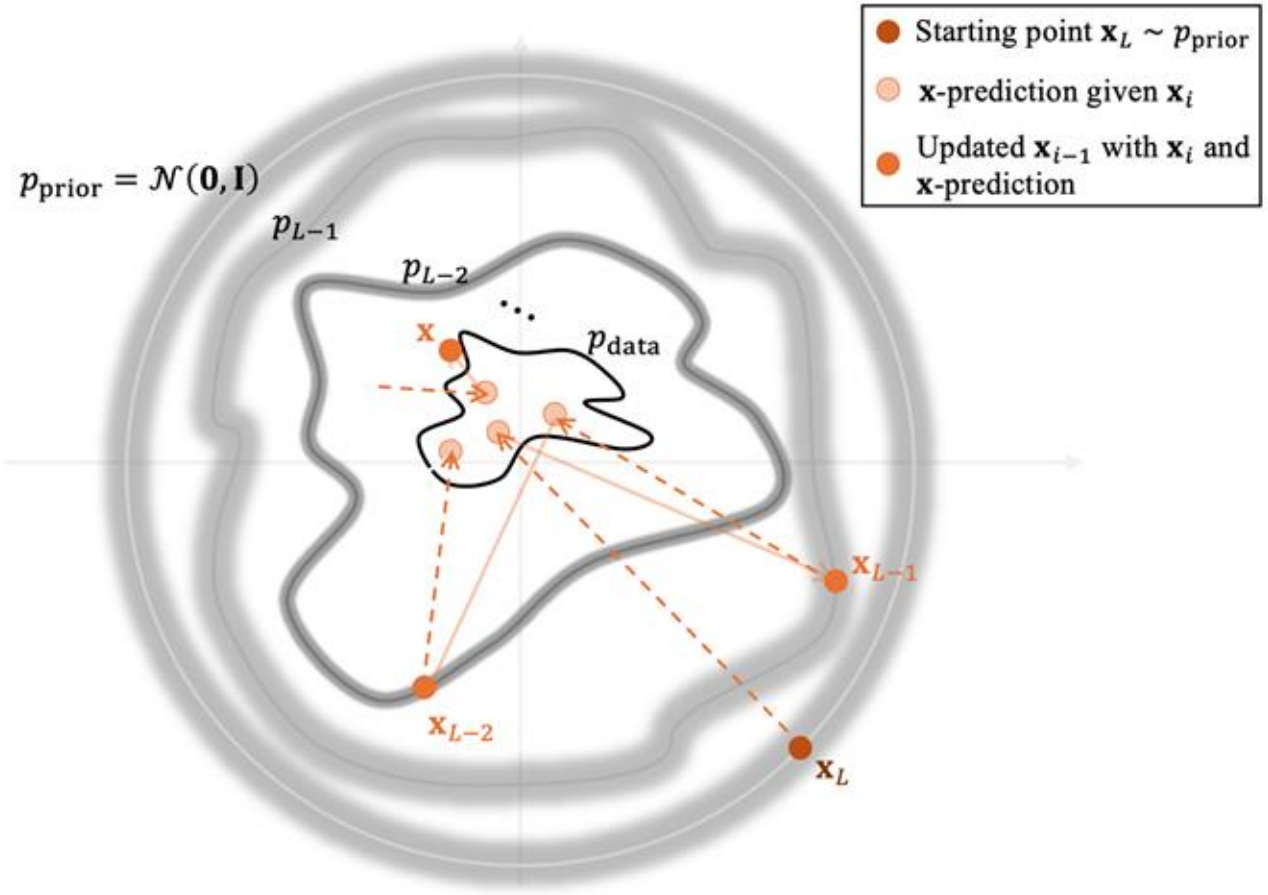
```
# 4. Predict the noise that was added
predicted_noise = model(x_t, t)
# 5. Calculate loss and optimize
loss = loss_fn(predicted_noise, noise)
```

Denoising Diffusion Probabilistic Models - Sampling


```
# 1. Start with pure noise x_T ~ N(0, I)
x = torch.randn((num_samples, 2))

# 2. Step backwards from T-1 down to 0
for i in reversed(range(T)):
    t = torch.full((num_samples,), i, dtype=torch.long)

    # Predict the noise using the model
    predicted_noise = model(x, t)

    # Fetch the schedule parameters for the current step t
    alpha = alphas[t].unsqueeze(-1)
    alpha_bar = alphas_cumprod[t].unsqueeze(-1)
    beta = betas[t].unsqueeze(-1)
```

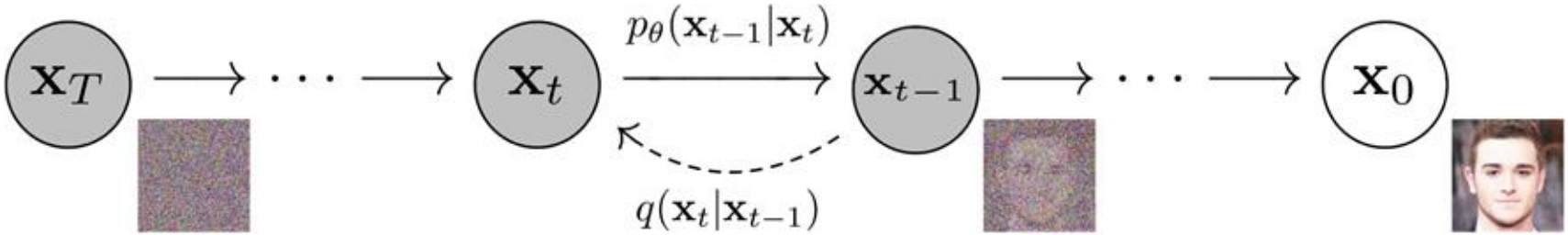
```
    # Reverse Step Formula:
    # x_{t-1} = 1/sqrt(alpha) * ( x_t - ((1-alpha) / sqrt(1-alpha_bar)) * predicted_noise ) + sqrt(beta) * z
    x = (1.0 / torch.sqrt(alpha)) * (x - ((1.0 - alpha) / torch.sqrt(1.0 - alpha_bar)) * predicted_noise) + torch.sqrt(beta) * z
```

Denoising Diffusion Probabilistic Models - Sampling



Denoising Diffusion Probabilistic Models - Summary

[Johnatan Ho et al., NeurIPS 2020]



Algorithm 1 Training

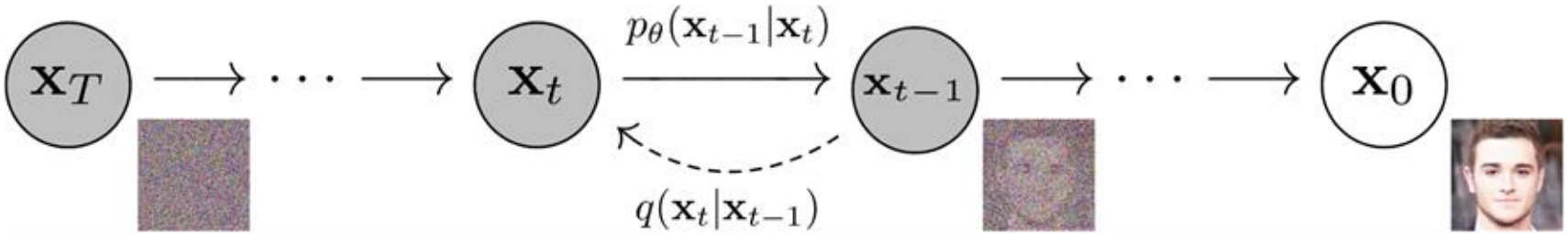
1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1, \dots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

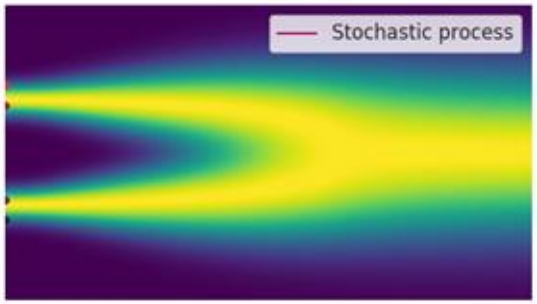
Algorithm 2 Sampling

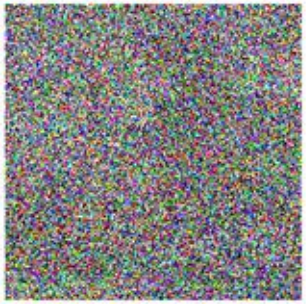
1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \dots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

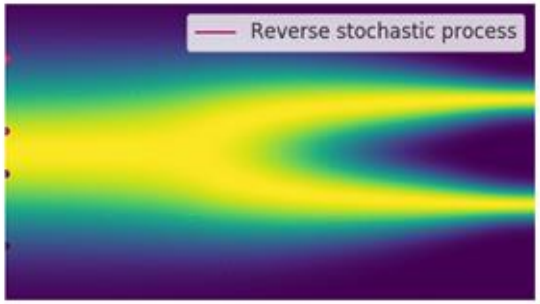
Denoising Diffusion Probabilistic Models - Definition

[Johnatan Ho et al., NeurIPS 2020]









Diffusion Models

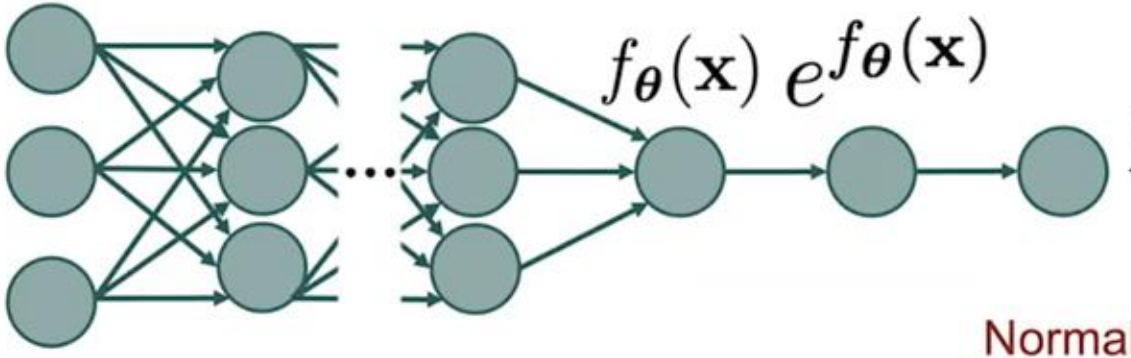
"Yet another path leading to the same model"

Score-based Generative Models

- Score function
- Score model

Key challenge

$\mathbf{x}$



$$Z_{\theta} = \int e^{f_{\theta}(\mathbf{x})} \, \mathrm{d}\mathbf{x}$$

While Diffusion Models were being developed at UC Berkeley, a team from Stanford University explored an alternative way to get around this fundamental problem:

$\longrightarrow$ Score-Based Generative Model

Working with score functions

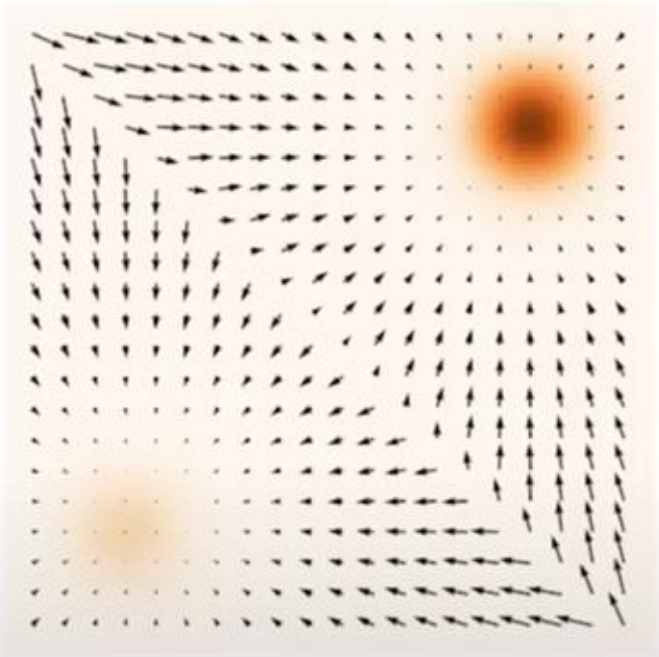
[Yang Song et Stefano Ermon. NeurIPS 2019; Song & Ermon. NeurIPS 2020; Song et al. ICLR 2021; Song et al. ICLR 2022]

$p(\mathbf{x})$

Probability density function

$\nabla_{\mathbf{x}} \log p(\mathbf{x})$

(Stein) score function



Score vs. density function

Score Based Generative Model - Definition

$$p(\mathbf{x}) = \frac{\tilde{p}(\mathbf{x})}{Z}, \qquad Z = \int \tilde{p}(\mathbf{x})\, \mathrm{d}\mathbf{x}.$$

While computing Z is intractable, the score depends only on $\tilde{p}$:

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}) = \nabla_{\mathbf{x}} \log \tilde{p}(\mathbf{x}) - \underbrace{\nabla_{\mathbf{x}} \log Z}_{=0} = \nabla_{\mathbf{x}} \log \tilde{p}(\mathbf{x}),$$

It is possible to sample from p only using the score function !

Langevin dynamics sampling

- Sample from $p(\mathbf{x})$ using only the score $\nabla_{\mathbf{x}} \log p(\mathbf{x})$
- Initialize $\mathbf{x}^0 \sim \pi(\mathbf{x})$
- Repeat for $t \leftarrow 1, 2, \cdots, T$

$$\mathbf{z}^t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$$

$$\mathbf{x}^t \leftarrow \mathbf{x}^{t-1} + \frac{\epsilon}{2} \nabla_{\mathbf{x}} \log p(\mathbf{x}^{t-1}) + \sqrt{\epsilon}\, \mathbf{z}^t$$

Stochasticity

- If $\epsilon \to 0$ and $T \to \infty$, we are guaranteed to have $\mathbf{x}^T \sim p(\mathbf{x})$

- Langevin dynamics + score estimation

$$\boldsymbol{s}_\theta(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p(\mathbf{x})$$

Langevin dynamics sampling

- Goal: sampling from $p(\mathbf{x})$ using only the score function $\nabla_{\mathbf{x}} \log p(\mathbf{x})$
- We initialize $x_0 \sim p_{prior} \equiv \mathcal{N}(0, I)$
- Repeat for $t \leftarrow 1, 2, 3, ..., T$

$$z_t \sim \mathcal{N}(0, I)$$

$$x_t = x_{t-1} + \frac{\epsilon}{2} \nabla_x \log p(x^{t-1}) + \sqrt{\epsilon} z_t$$

Stochasticity

- If $\epsilon \to 0$ and $T \to \infty$, we are guaranteed to have $x_T \sim p(x)$!
- Score Based Models combine langevin dynamics with score estimation:

$$s_\theta(x) \sim \nabla_x \log p(x)$$

Score models can be estimated from data

[Yang Song et Stefano Ermon. NeurIPS 2019; Song & Ermon. NeurIPS 2020; Song et al. ICLR 2012; Song et al. ICLR 2022]

Given: $\{\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_N\} \overset{\text{i.i.d.}}{\sim} p_{\text{data}}(\mathbf{x})$

Goal: $\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$

Score Model: $s_\theta(\mathbf{x}) : \mathbb{R}^d \to \mathbb{R}^d \approx \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$

Objective: How to compare two vector fields of scores?

$$\frac{1}{2}\mathbb{E}_{p_{\text{data}}(\mathbf{x})}[\|\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x}) - s_\theta(\mathbf{x})\|_2^2]$$

(Fisher divergence)

Score models can be estimated from data

[Yang Song et Stefano Ermon. NeurIPS 2019; Song & Ermon. NeurIPS 2020; Song et al. ICLR 2012; Song et al. ICLR 2022]

Given: $\{\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_N\} \overset{\text{i.i.d.}}{\sim} p_{\text{data}}(\mathbf{x})$

Goal: $\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$

Score Model: $s_\theta(\mathbf{x}) : \mathbb{R}^d \to \mathbb{R}^d \approx \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$

Objective: How to compare two vector fields of scores?

$$\frac{1}{2}\mathbb{E}_{p_{\text{data}}(\mathbf{x})}[\|\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x}) - s_\theta(\mathbf{x})\|_2^2]$$

(Fisher divergence)

Denoising score matching

- Sample a minibatch of datapoints $\{\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_n\} \sim p_{\text{data}}(\mathbf{x})$
- Sample a minibatch of perturbed datapoints $\{\tilde{\mathbf{x}}_1, \tilde{\mathbf{x}}_2, \cdots, \tilde{\mathbf{x}}_n\} \sim q_\sigma(\tilde{\mathbf{x}})$

$$\tilde{\mathbf{x}}_i \sim q_\sigma(\tilde{\mathbf{x}}_i \mid \mathbf{x}_i)$$

- Estimate the denoising score matching loss with empirical means

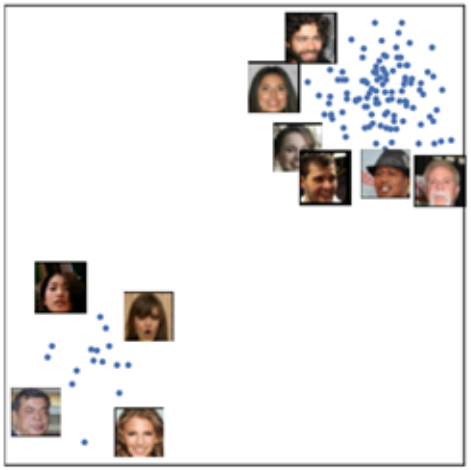
$$\frac{1}{2n}\sum_{i=1}^{n}[\|\boldsymbol{s}_\theta(\tilde{\mathbf{x}}_i) - \nabla_{\tilde{\mathbf{x}}} \log q_\sigma(\tilde{\mathbf{x}}_i \mid \mathbf{x}_i)\|_2^2]$$

- If Gaussian perturbation

$$\frac{1}{2n}\sum_{i=1}^{n}\left[\left\|\boldsymbol{s}_\theta(\tilde{\mathbf{x}}_i) + \frac{\tilde{\mathbf{x}}_i - \mathbf{x}_i}{\sigma^2}\right\|_2^2\right]$$

- Stochastic gradient descent
- Need to choose a very small σ!

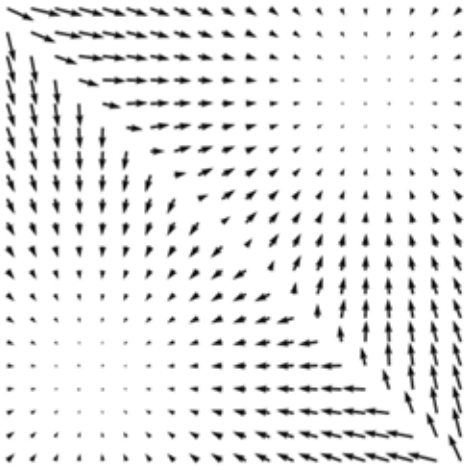
Score-Based Generative Models



Data samples

$$\{\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_N\} \overset{\text{i.i.d.}}{\sim} p(\mathbf{x})$$

score matching



Scores

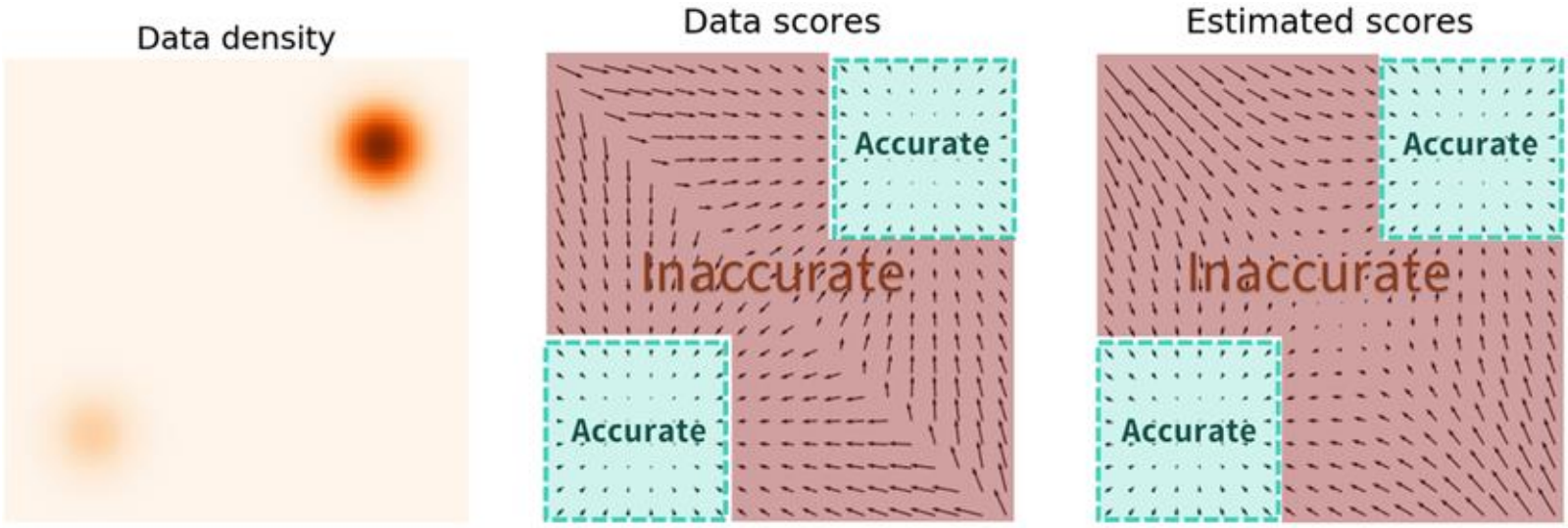
$$\mathbf{s}_\theta(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p(\mathbf{x})$$

Langevin dynamics

New samples

Any pitfalls ?

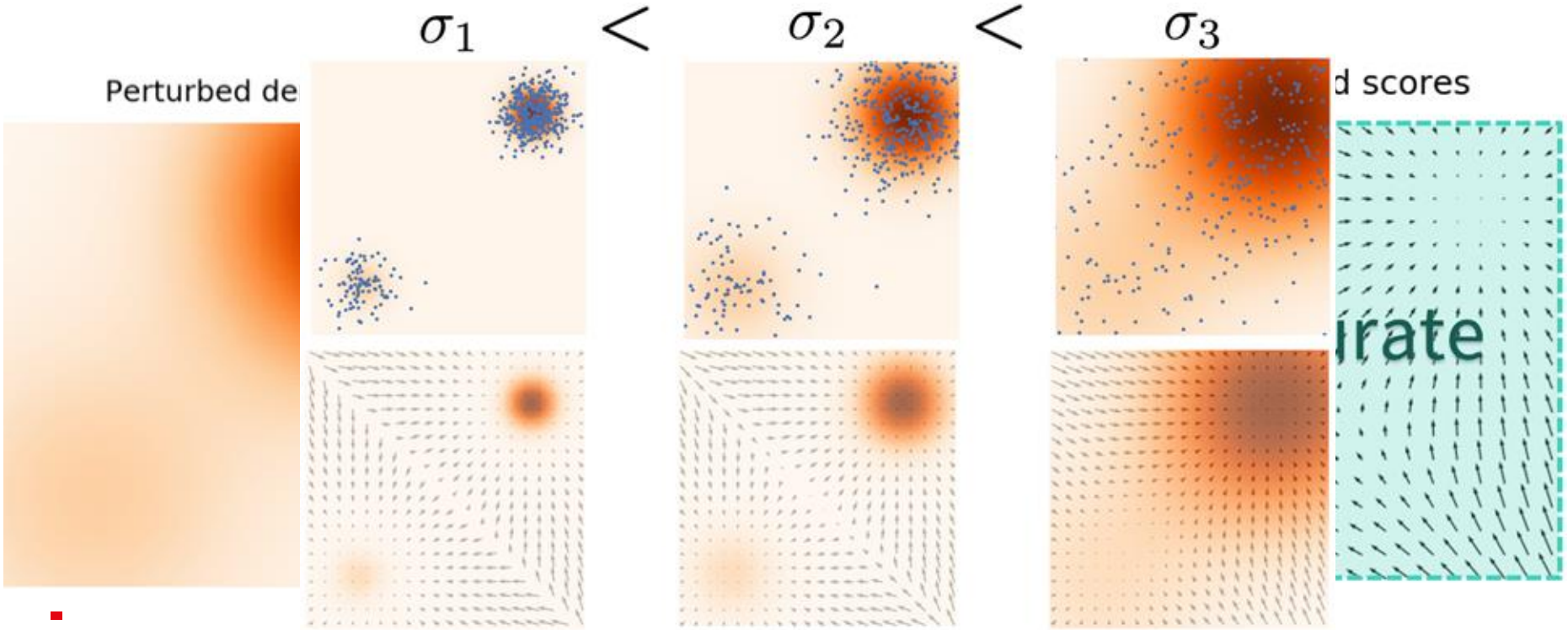
Score-Based Generative Models



Inaccurate Estimation in low-density regions

Score-Based Generative Models - Remedy

A solution is to perturb the data points with different degree of noise and train score-based models on the noisy data points instead.



Score based Generative Models - Training


```
# 1. Sample a batch of clean data
idx = torch.randint(0, len(data), (batch_size,))
x = data[idx]
# 2. Randomly select a sigma for each data point from our 10 sigmas
sigma_idx = torch.randint(0, num_sigmas, (batch_size,))
batch_sigmas = sigmas[sigma_idx]
# 3. Add Gaussian noise to the data
noise = torch.randn_like(x)
perturbed_x = x + noise * batch_sigmas.unsqueeze(-1)
# 4. Calculate the TARGET score
# For Gaussian noise, the true score is -(perturbed_x - x) / sigma^2
target_score = -(perturbed_x - x) / (batch_sigmas.unsqueeze(-1) ** 2)
# 5. Predict the score
predicted_score = model(perturbed_x, batch_sigmas)
# 6. Denoising Score Matching Loss
loss = (batch_sigmas.unsqueeze(-1) ** 2) * (predicted_score - target_score)**2
```

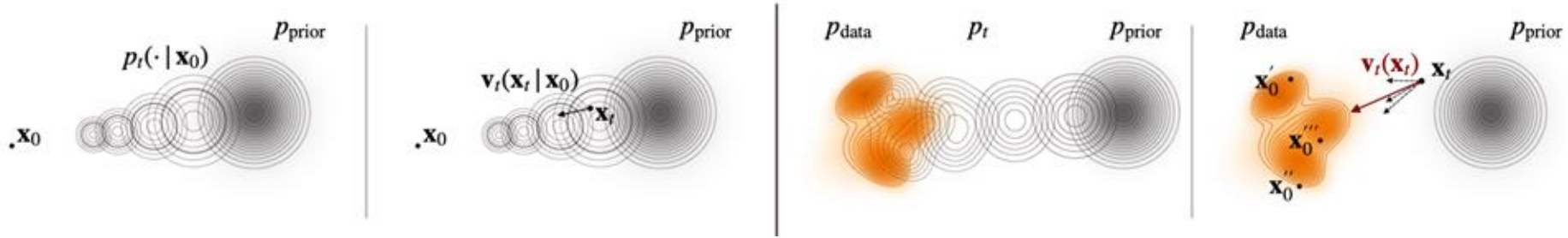
Score based Generative Models - Sampling


```
# 1. Initialize random noise from the largest sigma distribution
# Prior is N(0, sigma_max^2 I)
x = torch.randn((num_samples, 2)) * sigmas[0]
# 2. Iterate down the 10 sigmas (from largest noise to smallest)
for i, sigma in enumerate(sigmas):
    alpha_i = base_step_size * (sigma / sigmas[-1]) ** 2

    # Create a batch of the current sigma for the model
    batch_sigma = torch.full((num_samples,), sigma.item())
    # Run Langevin dynamics for a fixed number of steps at this noise scale
    for step in range(steps_per_sigma):
        # Predict the score
        score = model(x, batch_sigma)
        # Sample stochastic noise z
        z = torch.randn_like(x)
        # Langevin update formula:
        # x_{t+1} = x_t + (alpha_i / 2) * score + sqrt(alpha_i) * z
        x = x + (alpha_i / 2) * score + torch.sqrt(torch.tensor(alpha_i)) * z
```

"Yet another path leading to the same model"

Conditional Flow Matching

Conditional Flow Matching - Ordinary Diff. Eq.



$$\frac{\mathrm{d}\mathbf{x}(t)}{\mathrm{d}t} = \mathbf{v}_t\left(\mathbf{x}(t)\right). \quad \Longrightarrow \quad x_{t+\Delta_t} = x_t + \Delta_t \cdot \frac{\mathrm{d}x}{\mathrm{d}t}$$

- Addressing the intractability of the true marginal velocity by constructing an artificial but tractable process:
  - We introduce a conditioning variable **Z** and "design" either :
    - $\mathbf{v}_t(\mathbf{x}_t|\mathbf{z})$

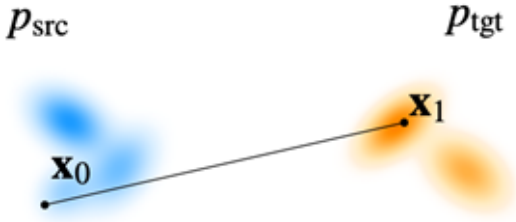
      $p_t(\mathbf{x}_t|\mathbf{z})$

    So that **v** has a closed form, or *p* can be sampled easily.

A path between two arbitrary distributions: src → tgt

- In particular when p_t is Gaussian $p_t(\mathbf{x}_t) = \mathcal{N}\left(\mathbf{x}_t; \boldsymbol{\mu}(t), \sigma^2(t)\mathbf{I}\right),$

$$\mathbf{v}_t(\mathbf{x}) = \frac{\sigma'(t)}{\sigma(t)}\left(\mathbf{x} - \boldsymbol{\mu}(t)\right) + \boldsymbol{\mu}'(t).$$



$$\boxed{\mathbf{v}_t(\mathbf{x}_t|\mathbf{x}_1) = b'_t\mathbf{x}_1 + \frac{a_t}{a_t}\left(\mathbf{x}_t - b_t\mathbf{x}_1\right) = a'_t\mathbf{x}_0 + b'_t\mathbf{x}_1.}$$

CFM - Training


```
# 1. Sample clean data (Target endpoint: t=1)
idx = torch.randint(0, len(data), (batch_size,))
x_1 = data[idx]
# 2. Sample standard Gaussian noise (Source endpoint: t=0)
x_0 = torch.randn_like(x_1)
# 3. Sample random times t uniformly between 0 and 1
t = torch.rand((batch_size, 1))
# 4. Interpolate to find the exact state x_t at time t (Straight Line / OT path)
x_t = (1 - t) * x_0 + t * x_1
# 5. The true velocity is simply the vector pointing from x_0 to x_1
target_velocity = x_1 - x_0
# 6. Predict the velocity and compute MSE loss
predicted_velocity = model(x_t, t)
loss = loss_fn(predicted_velocity, target_velocity)
```

CFM - Sampling (Euler method)


```
# Calculate the fixed time step size (dt) to go from t=0 to t=1
dt = 1.0 / steps
# Iterate forward through time from 0 to 1
for i in range(steps):
    # Create a batched tensor of the current time t
    t = torch.full((num_samples,), i * dt)
    # Predict the velocity field at the current position and time
    v = model(x, t)
    # Apply the Euler step: Move x along the velocity vector by dt
    x = x + v * dt
```

Conditional Flow Matching - Link to Score

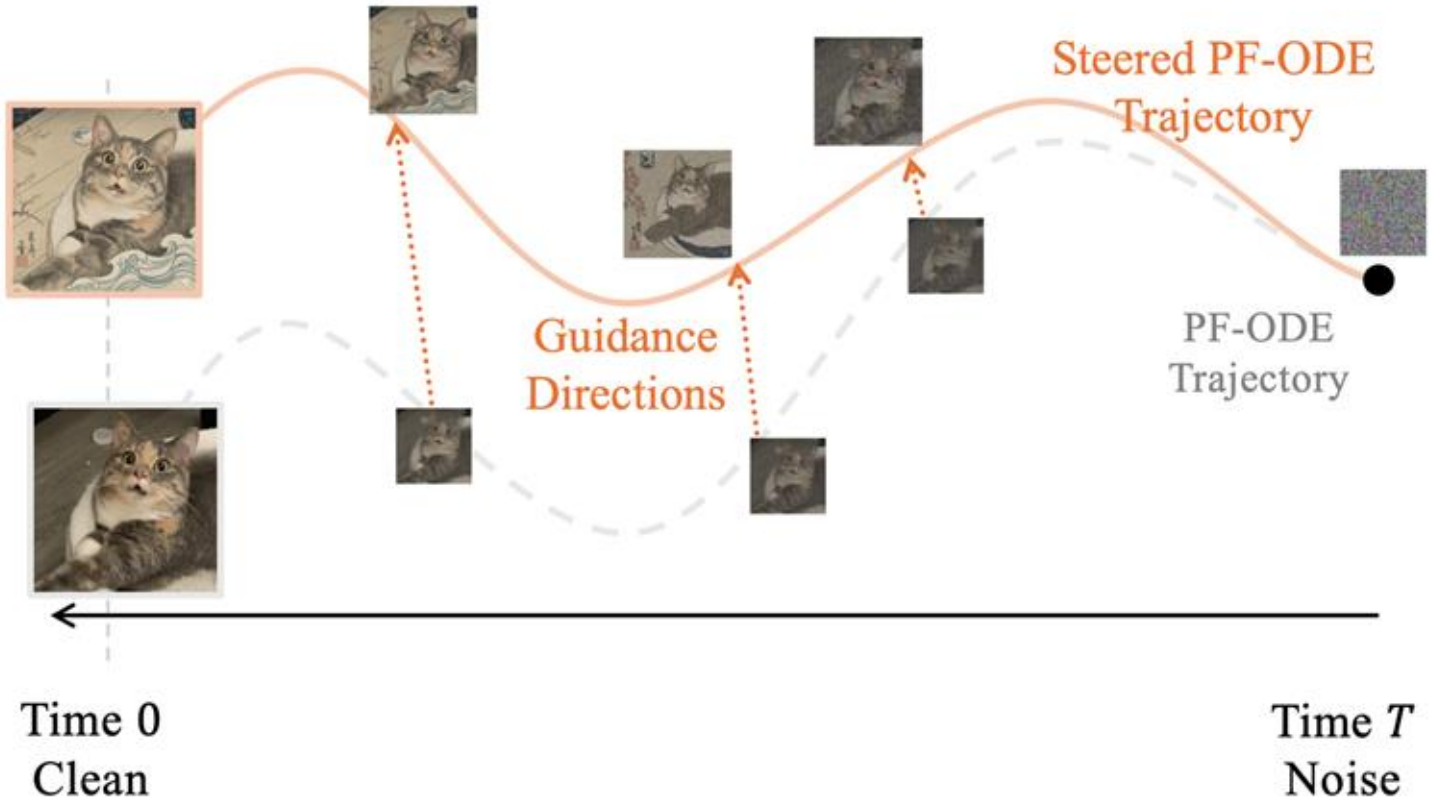
$$p(\mathbf{x}_{t+\Delta t}|\mathbf{x}_t) = \mathcal{N}\left(\mathbf{x}_{t+\Delta t}; \mathbf{x}_t + \mathbf{f}(\mathbf{x}_t, t)\Delta t, g^2(t)\Delta t\mathbf{I}\right),$$

$$\mathbf{v}_t(\mathbf{x}) := f(t)\mathbf{x} - \frac{1}{2}g^2(t)\nabla_{\mathbf{x}} \log p_t(\mathbf{x}),$$

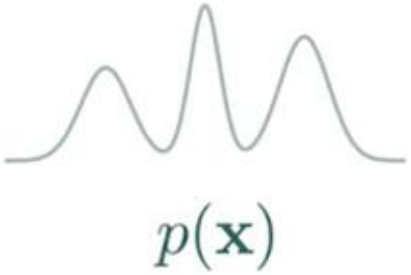
Controllable Generation

- Control the generation process

Conditional Flow Matching - Link to Score



Control the generation process

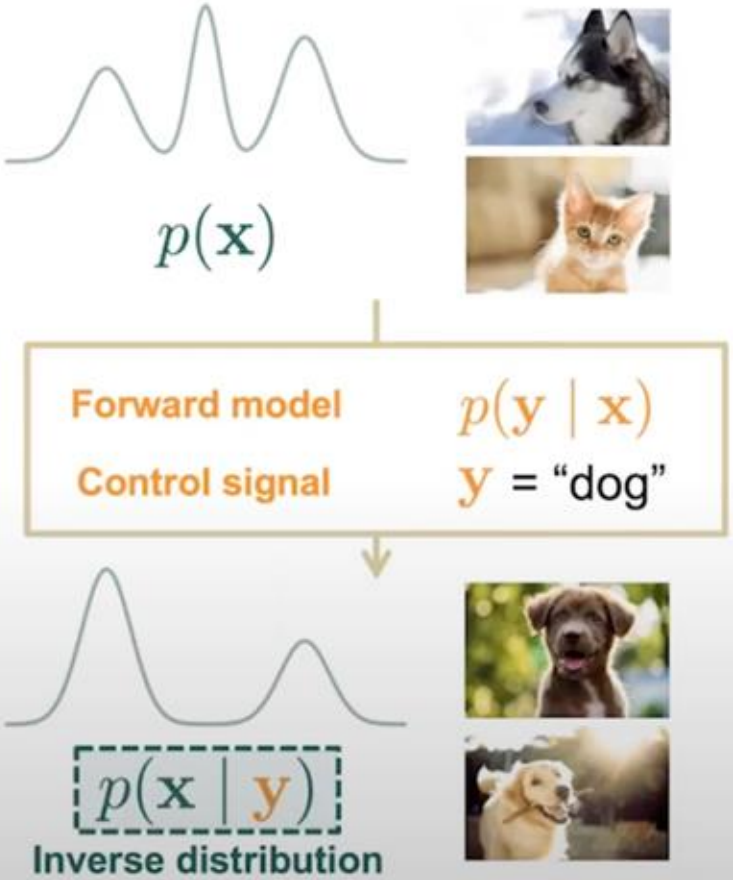


Control the generation process

$p(\mathbf{x})$

Forward model	$p(\mathbf{y} \mid \mathbf{x})$
Control signal	$\mathbf{y}$ = “dog”

Control the generation process



Control the generation process

Bayes' rule:

$$p(\mathbf{x} \mid \mathbf{y}) = \frac{p(\mathbf{x})\,p(\mathbf{y} \mid \mathbf{x})}{p(\mathbf{y})}$$

Bayes' rule for score functions:

Control the generation process

Bayes' rule:

$$p(\mathbf{x} \mid \mathbf{y}) = \frac{p(\mathbf{x})\,p(\mathbf{y} \mid \mathbf{x})}{p(\mathbf{y})}$$

Bayes' rule for score functions:

$$\nabla_{\mathbf{x}} \log p(\mathbf{x} \mid \mathbf{y}) = \nabla_{\mathbf{x}} \log p(\mathbf{x}) + \nabla_{\mathbf{x}} \log p(\mathbf{y} \mid \mathbf{x}) - \nabla_{\mathbf{x}} \log p(\mathbf{y})$$

$$= \nabla_{\mathbf{x}} \log p(\mathbf{x}) + \nabla_{\mathbf{x}} \log p(\mathbf{y} \mid \mathbf{x}), \quad \text{with } \nabla_{\mathbf{x}} \log p(\mathbf{y}) = 0$$

Unconditional score $\approx s_\theta(\mathbf{x})$

Forward model

Control the generation process

Bayes' rule:

$$p(\mathbf{x} \mid \mathbf{y}) = \frac{p(\mathbf{x})\,p(\mathbf{y} \mid \mathbf{x})}{p(\mathbf{y})}$$

Bayes' rule for score functions:

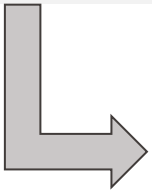
$$\nabla_{\mathbf{x}} \log p(\mathbf{x} \mid \mathbf{y}) = \nabla_{\mathbf{x}} \log p(\mathbf{x}) + \nabla_{\mathbf{x}} \log p(\mathbf{y} \mid \mathbf{x}) - \underbrace{\nabla_{\mathbf{x}} \log p(\mathbf{y})}_{\mathbf{0}}$$

$$= \nabla_{\mathbf{x}} \log p(\mathbf{x}) + \nabla_{\mathbf{x}} \log p(\mathbf{y} \mid \mathbf{x})$$

Plug in different forward models for the same score model

Classifier Guidance

$$\nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t|\mathbf{c}) = \underbrace{\nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t)}_{\text{unconditional direction}} + \underbrace{\nabla_{\mathbf{x}_t} \log p_t(\mathbf{c}|\mathbf{x}_t)}_{\text{guidance direction}}.$$



You directly replace your unconditional model prediction by your conditional estimate at each step during sampling.

Inconvenience: You need a classifier for each time *t* …….

Classifier-Free Guidance - CFG

$$\epsilon_\theta(x_t, t, c),\ x_\theta(x_t, t, c),\ s_\theta(x_t, t, c),\ v_\theta(x_t, t, c)$$

- First, when training the model, we add the condition on top of the temporal condition.

```
predicted_noise = model(x_t, t, c)
```

$$w \cdot \epsilon_\theta(x_t, t, c) + (1 - w) \cdot \epsilon_\theta(x_t, t, \varnothing),$$
where w is the guidance strength.

Classifier-Free Guidance - CFG

$$\epsilon_\theta(x_t, t, c),\ x_\theta(x_t, t, c),\ s_\theta(x_t, t, c),\ v_\theta(x_t, t, c)$$

- Any downside?

To train any of those you need **c** !!!

Requires huge datasets fully labelled !

Classifier-Free Guidance - Sampling


```
# 1. Start with pure random noise x_T
x = torch.randn((num_samples, *image_shape), device=device)
null_condition = torch.zeros_like(condition)
# 3. Step backwards from T-1 down to 0
for i in reversed(range(T)):
    # Create the time tensor
    t = torch.full((num_samples,), i, device=device, dtype=torch.long)
    # ------------------------------------------------------------------
    # THE CFG "BATCH DOUBLING" TRICK
    # ------------------------------------------------------------------
    # Instead of running the model twice, we concatenate the inputs.
    # Top half will be conditional, bottom half will be unconditional.
    x_doubled = torch.cat([x, x], dim=0)
    t_doubled = torch.cat([t, t], dim=0)
    c_doubled = torch.cat([condition, null_condition], dim=0)
    # 4. Predict the noise for BOTH simultaneously
    predicted_noise = model(x_doubled, t_doubled, c_doubled)
    # 5. Split the predictions back into conditional and unconditional
    eps_cond, eps_uncond = predicted_noise.chunk(2, dim=0)
    # 6. Apply the Classifier-Free Guidance Formula
    # Formula: eps_guided = eps_uncond + w * (eps_cond - eps_uncond)
    eps_guided = eps_uncond + w * (eps_cond - eps_uncond)
```

Image Colorization

Ground truth

Gray images

$\mathbf{y}$

Colorized images

$\mathbf{x} \mid \mathbf{y}$

Forward model can be directly specified

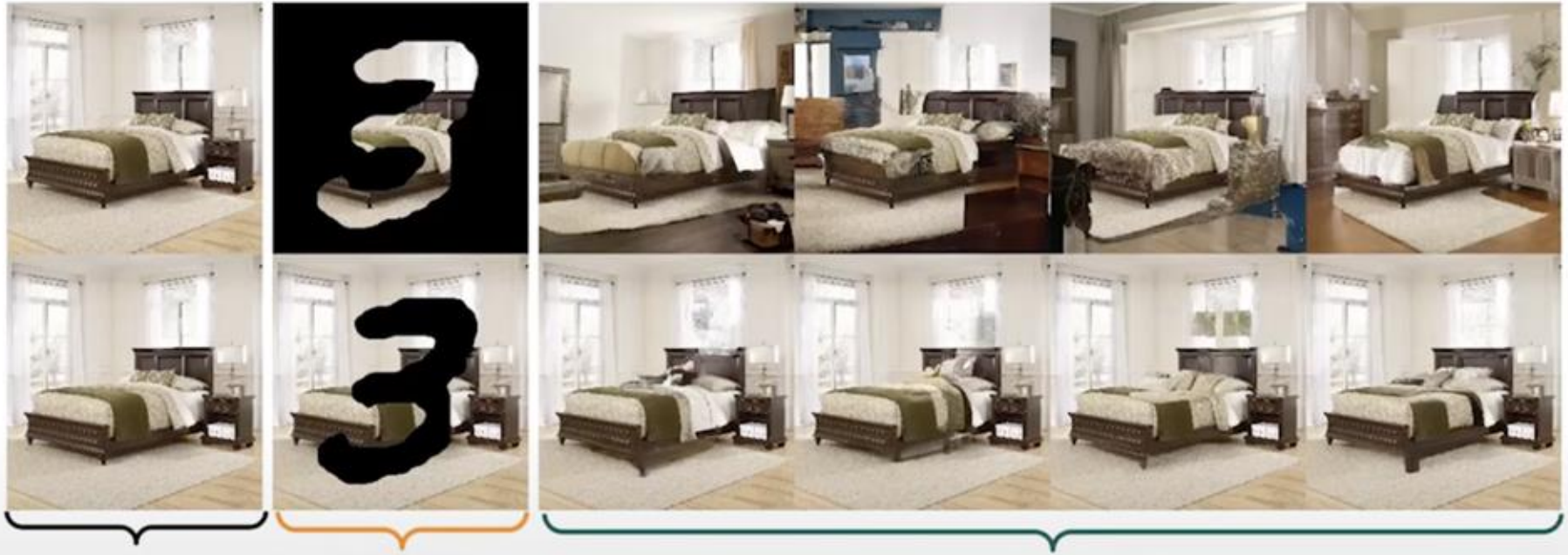
Class-conditional generation

- $\mathbf{y}$ is the **class label**
- $p_t(\mathbf{y} \mid \mathbf{x})$ is a time-dependent classifier (classifier-guidance)

class: bird

class: deer

Image inpainting



Ground truth | Masked images | Inpainted images

$\mathbf{y}$

$\mathbf{x} \mid \mathbf{y}$

Forward model can be directly specified

Language-guided image generation

$\mathbf{y}$

$\mathbf{x} \mid \mathbf{y}$

(Prompt)
Treehouse in the style of Studio Ghibli animation

Forward model

$p(\mathbf{y} \mid \mathbf{x})$

is an image captioning neural network.

[Work by @danielrussruss]

EPFL VITA

Thank you for your attention